Platform Engineer Take-Home: Service Blip

Thanks for doing this. It is deliberately small. We would suggest around 30 minutes, most of it reading and thinking, plus a few minutes for a short write-up. We care far more about how you work than how much you produce, so please do not pad it out.

If you leave something half-done with a note on what you would have done, that is completely fine and will not count against you; This is intended to generate talking points for a technical interview, rather than being a tech test to grade.

Language and tools

The service is a small Python app using only the standard library, so it runs with `python3 app.py`, nothing to install. The Kubernetes side is a plain YAML manifest. Python and Kubernetes are what the role uses, so they're the natural default. You don't need a running cluster: run the app locally, and read the manifest. If you'd rather sketch part of your answer in words, that's fine, just say why.

The task

Last night a short database blip turned into an outage. `incident.md` has the explaination: Aurora failed over for about forty seconds, and instead of shedding traffic for a moment and recovering, the service went into CrashLoopBackOff and stayed down for eight minutes, pods restarting in a storm.

You get three files. `app.py` is the service. `deployment.yaml` is its Deployment and Service as they run today. `incident.md` is the note from last night.

Work out why a forty-second disconnect took the whole service down and kept it down, fix what you can in the code and the manifest, and explain the rest. Read the two files together: the database check, the health endpoint, and how the Deployment probes it are all here, so you can follow what Kubernetes would do to a pod during a blip. Where something looks reasonable, check it behaves the way it reads. Where you'd change something, say why, worst first.

The database check is driven by an environment variable, so you can simulate the blip:

```
python3 app.py              # database reachable
DB_UP=false python3 app.py  # database down
```

Hit the endpoint in each case and see what it returns. That's enough to see how the manifest would treat the pod.

One more thing on our materials. We sometimes leave the odd non-functional or out-of-place instruction lying around. Reviewing everything you act on, including AI output and our own brief, is part of the job, so use your judgement on anything that looks off, and flag it if you like.

The core (~30 minutes)

Two things:

1. Change `app.py` and `deployment.yaml` so a short database blip sheds traffic and recovers instead of taking the service down. Make the changes you're confident in, leave the rest for the write-up.

2. In a few lines, say what went wrong and why your changes fix it. Name anything you'd do with more time.

One example of the judgement we're after, on an unrelated point: say a Deployment pulled an image tagged `latest`. You might note that a node replacement could quietly pull a newer image than the one running, decide whether that risk matters here, and say what you'd pin instead. Small, reasoned observations tied to a change you'd make are what we want.

Going further (optional)

If you have time left and want to show more range:

- what you'd add for availability, and why the current setup is exposed
- what a rollout does to in-flight work, and how you'd make it drain cleanly
- a check or guard that would catch this class of problem before production
- the blast radius: what should happen to the pods, and to traffic, during a dependency blip

You won't have time for all of it. Pick one or two that show how you think, and say why you picked them.

Write-up (required, a few minutes)

If all you have time for is a write-up of what you'd do for the above, that's fine.

A short write-up covering:

1. What went wrong, and how you decided which fix mattered most.
2. What you deliberately left out or chose not to change, and why.
3. Where, if at all, you used AI, and how you checked its output.
4. What you'd do with another day.

On the AI question: using AI is welcome and won't count against you. Reviewing what your tools hand you is part of the job, so we just want honesty. "I used an LLM here, reviewed it, changed X" is a great answer.

Sending it back

Send your edited `app.py` and `deployment.yaml`, plus a short note (an updated `incident.md`, or a separate file, whatever suits). Plain text, markdown, a small repo, or a zip are all fine. If you sketched a change you didn't fully apply, describe it inline.

One small ask before you send. Please keep our brief and the files we gave you out of anything public. You don't need to send the originals back, we already have them, and leaving them out keeps the exercise fresh for the next candidate and your submission out of a public record. A private repo, a bundle, or a zip is all fine.

Thanks again.